

Pima Indians Diabetes Dataset

K-Means Clustering And PCA

1. Import Libraries

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score
from sklearn.decomposition import PCA

import warnings
warnings.filterwarnings("ignore")
```

2. Load Dataset

```
# If using Google Colab, upload diabetes.csv first.

try:
    df = pd.read_csv("diabetes.csv")
except FileNotFoundError:
    from google.colab import files
    uploaded = files.upload()
    df = pd.read_csv("diabetes.csv")

print("=" * 70)
print("PIMA INDIANS DIABETES DATASET")
print("=" * 70)

print("\nDataset Shape:", df.shape)

print("\nFirst 5 Rows:")
print(df.head())

print("\nDataset Information:")
print(df.info())

print("\nStatistical Summary:")
print(df.describe())
```

3. Check Missing Values

```
print("\nMissing Values:")
print(df.isnull().sum())
```

4. Handle Invalid Values

```
# Zero values in these medical variables are considered invalid/missing.

invalid_columns = [
    "Glucose",
    "BloodPressure",
    "SkinThickness",
    "Insulin",
    "BMI"
]

df_clean = df.copy()

print("\nInvalid Zero Values:")
for col in invalid_columns:
    print(col, ":", (df_clean[col] == 0).sum())

# Replace zeros with NaN
for col in invalid_columns:
    df_clean[col] = df_clean[col].replace(0, np.nan)

# Replace missing values with median
for col in invalid_columns:
    df_clean[col] = df_clean[col].fillna(df_clean[col].median())

print("\nMissing Values After Imputation:")
print(df_clean.isnull().sum())
```

5. Select Features

```
# Outcome is excluded because this is unsupervised learning.

features = [
    "Pregnancies",
    "Glucose",
    "BloodPressure",
    "SkinThickness",
    "Insulin",
    "BMI",
    "DiabetesPedigreeFunction",
    "Age"
]
```

```
X = df_clean[features]

print("\nFeatures Used for Clustering:")
print(features)
```

6. Standardize Features

```
scaler = StandardScaler()

X_scaled = scaler.fit_transform(X)

print("\nStandardization Completed.")

X_scaled_df = pd.DataFrame(
    X_scaled,
    columns=features
)

print("\nStandardized Data:")
print(X_scaled_df.head())
```

7. Elbow Method

```
inertias = []
k_values = range(2, 11)

for k in k_values:
    kmeans = KMeans(
        n_clusters=k,
        random_state=42,
        n_init=10
    )

    kmeans.fit(X_scaled)

    inertias.append(kmeans.inertia_)

# Plot Elbow Curve
plt.figure(figsize=(8, 5))

plt.plot(
    k_values,
    inertias,
    marker="o",
    color="blue",
    linewidth=2
)
```

```
plt.xlabel("Number of Clusters (k)")
plt.ylabel("Inertia / WCSS")
plt.title("Elbow Method")

plt.xticks(k_values)
plt.grid(True, alpha=0.3)

plt.show()
```

8. Silhouette Score

```
silhouette_scores = []

for k in range(2, 11):
    kmeans = KMeans(
        n_clusters=k,
        random_state=42,
        n_init=10
    )

    labels = kmeans.fit_predict(X_scaled)

    score = silhouette_score(
        X_scaled,
        labels
    )

    silhouette_scores.append(score)

# Create results table
silhouette_results = pd.DataFrame({
    "Number of Clusters": range(2, 11),
    "Silhouette Score": silhouette_scores
})

print("\nSilhouette Scores:")
print(silhouette_results)

# Plot Silhouette Scores
plt.figure(figsize=(8, 5))

plt.plot(
    silhouette_results["Number of Clusters"],
    silhouette_results["Silhouette Score"],
    marker="o",
    color="green",
    linewidth=2
```

```

)

plt.xlabel("Number of Clusters (k)")
plt.ylabel("Silhouette Score")
plt.title("Silhouette Score for Different k Values")

plt.grid(True, alpha=0.3)

plt.show()

# Select k with highest silhouette score
best_k = int(
    silhouette_results.loc[
        silhouette_results["Silhouette Score"].idxmax(),
        "Number of Clusters"
    ]
)

best_score = silhouette_results["Silhouette Score"].max()

print("\nBest Number of Clusters:", best_k)
print("Best Silhouette Score:", round(best_score, 4))

```

9. Final K-Means Clustering

```

kmeans = KMeans(
    n_clusters=best_k,
    random_state=42,
    n_init=10
)

clusters = kmeans.fit_predict(X_scaled)

df_clean["Cluster"] = clusters

print("\nK-Means Clustering Completed.")

print("\nCluster Sizes:")
print(
    df_clean["Cluster"]
    .value_counts()
    .sort_index()
)

```

10. Cluster Characteristics

```

cluster_summary = df_clean.groupby("Cluster")[features].mean()

print("\nMean Feature Values for Each Cluster:")

```

```

print(cluster_summary)

# Heatmap of cluster means
plt.figure(figsize=(12, 6))

sns.heatmap(
    cluster_summary,
    annot=True,
    fmt=".2f",
    cmap="coolwarm"
)

plt.title("Mean Feature Values by Cluster")
plt.xlabel("Features")
plt.ylabel("Cluster")

plt.tight_layout()
plt.show()

```

11. Standardized Cluster Centers

```

cluster_centers = pd.DataFrame(
    kmeans.cluster_centers_,
    columns=features
)

print("\nStandardized Cluster Centers:")
print(cluster_centers)

plt.figure(figsize=(12, 6))

sns.heatmap(
    cluster_centers,
    annot=True,
    fmt=".2f",
    cmap="coolwarm",
    center=0
)

plt.title("Standardized Cluster Centers")
plt.xlabel("Features")
plt.ylabel("Cluster")

plt.tight_layout()
plt.show()

```

12. Compare Clusters With Outcome

```
cluster_outcome = pd.crosstab(
    df_clean["Cluster"],
    df_clean["Outcome"]
)

print("\nCluster vs Outcome:")
print(cluster_outcome)

# Percentage distribution
cluster_outcome_percentage = pd.crosstab(
    df_clean["Cluster"],
    df_clean["Outcome"],
    normalize="index"
) * 100

print("\nCluster vs Outcome Percentage:")
print(cluster_outcome_percentage.round(2))

# Plot Outcome Distribution
cluster_outcome_percentage.plot(
    kind="bar",
    figsize=(8, 5),
    color=["steelblue", "tomato"]
)

plt.xlabel("Cluster")
plt.ylabel("Percentage")
plt.title("Diabetes Outcome Distribution Within Clusters")

plt.legend(
    title="Outcome",
    labels=["No Diabetes", "Diabetes"]
)

plt.xticks(rotation=0)
plt.tight_layout()

plt.show()
```

13. Principal Component Analysis (PCA)

```
pca = PCA(n_components=2)

X_pca = pca.fit_transform(X_scaled)

pca_df = pd.DataFrame(
```

```

X_pca,
columns=["PC1", "PC2"]
)

pca_df["Cluster"] = clusters
pca_df["Outcome"] = df_clean["Outcome"].values

```

14. PCA Explained Variance

```

explained_variance = pca.explained_variance_ratio_

print("\nPCA Explained Variance:")

print(
    "PC1:",
    round(explained_variance[0] * 100, 2),
    "%"
)

print(
    "PC2:",
    round(explained_variance[1] * 100, 2),
    "%"
)

print(
    "Total:",
    round(explained_variance.sum() * 100, 2),
    "%"
)

# Plot Explained Variance
plt.figure(figsize=(7, 5))

plt.bar(
    ["PC1", "PC2"],
    explained_variance * 100,
    color=["steelblue", "orange"]
)

plt.ylabel("Explained Variance (%)")
plt.title("PCA Explained Variance")

plt.show()

```

15. PCA Visualization Of Clusters

```

plt.figure(figsize=(10, 7))

```

```

sns.scatterplot(
    data=pca_df,
    x="PC1",
    y="PC2",
    hue="Cluster",
    palette="Set1",
    s=70,
    alpha=0.75
)

plt.title("K-Means Clusters in PCA 2D Space")
plt.xlabel("Principal Component 1")
plt.ylabel("Principal Component 2")

plt.legend(title="Cluster")

plt.grid(True, alpha=0.2)
plt.tight_layout()

plt.show()

```

16. PCA Visualization Using Diabetes Outcome

```

plt.figure(figsize=(10, 7))

sns.scatterplot(
    data=pca_df,
    x="PC1",
    y="PC2",
    hue="Outcome",
    palette={
        0: "steelblue",
        1: "red"
    },
    s=70,
    alpha=0.75
)

plt.title("PCA Representation Colored by Diabetes Outcome")
plt.xlabel("Principal Component 1")
plt.ylabel("Principal Component 2")

plt.legend(
    title="Outcome",
    labels=["No Diabetes", "Diabetes"]
)

plt.grid(True, alpha=0.2)
plt.tight_layout()

```

```
plt.show()
```

17. PCA Feature Loadings

```
loadings = pd.DataFrame(  
    pca.components_.T,  
    columns=["PC1", "PC2"],  
    index=features  
)  
  
print("\nPCA Feature Loadings:")  
print(loadings)  
  
# Plot PCA loadings  
plt.figure(figsize=(8, 6))  
  
sns.heatmap(  
    loadings,  
    annot=True,  
    cmap="coolwarm",  
    center=0  
)  
  
plt.title("PCA Feature Loadings")  
  
plt.tight_layout()  
plt.show()
```

18. Final Results

```
print("\n")  
print("=" * 70)  
print("FINAL RESULTS")  
print("=" * 70)  
  
print("\nOriginal Dataset Shape:", df.shape)  
  
print("\nNumber of Features Used:", len(features))  
  
print("\nSelected Number of Clusters:", best_k)  
  
print(  
    "Silhouette Score:",  
    round(best_score, 4)  
)  
  
print("\nPCA Variance Explained:")
```

```
print(
    "PC1:",
    round(explained_variance[0] * 100, 2),
    "%"
)

print(
    "PC2:",
    round(explained_variance[1] * 100, 2),
    "%"
)

print(
    "Total:",
    round(explained_variance.sum() * 100, 2),
    "%"
)

print("\nCluster Sizes:")
print(
    df_clean["Cluster"]
    .value_counts()
    .sort_index()
)

print("\nCluster Mean Values:")
print(cluster_summary.round(2))

print("\nCluster vs Outcome:")
print(cluster_outcome)

print("\nExperiment Completed Successfully!")

print("=" * 70)
```